GUI

15-3-26

Aim – Develop a python gui program using only tkinter library to perform various unit conversions suck as currency(Rupees to dollars), temperature (Celsius to Fahrenheit), and length (Inches to Feet). The application should include input fields for the values, dropdown menus or buttons select the type of conversion, and labels to display the result

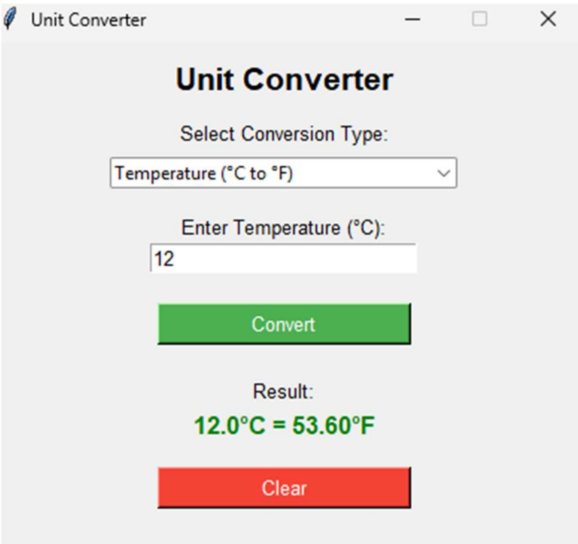
Theory – Graphical User Interface (GUI) programming in Python can be done using the Tkinter library. Tkinter provides tools to create windows, labels, buttons, and input fields. In this program, a student form interface is created where users enter details like name, roll number, branch, and email. The program retrieves input values from entry widgets and validates them. Message boxes are used to display warnings for incomplete data and confirmations when the form is successfully submitted.

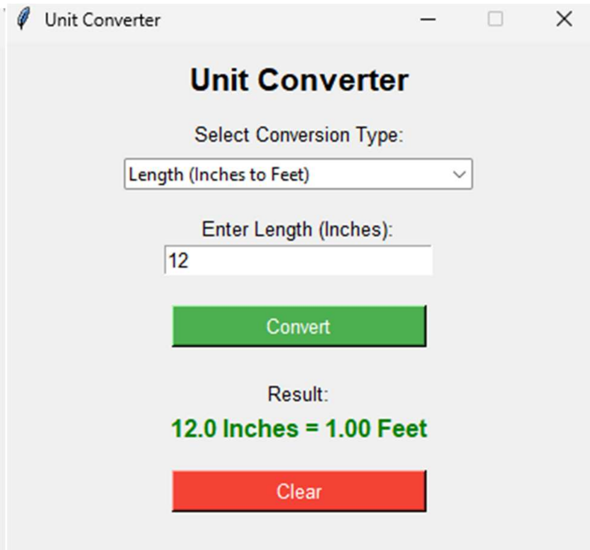
A1.

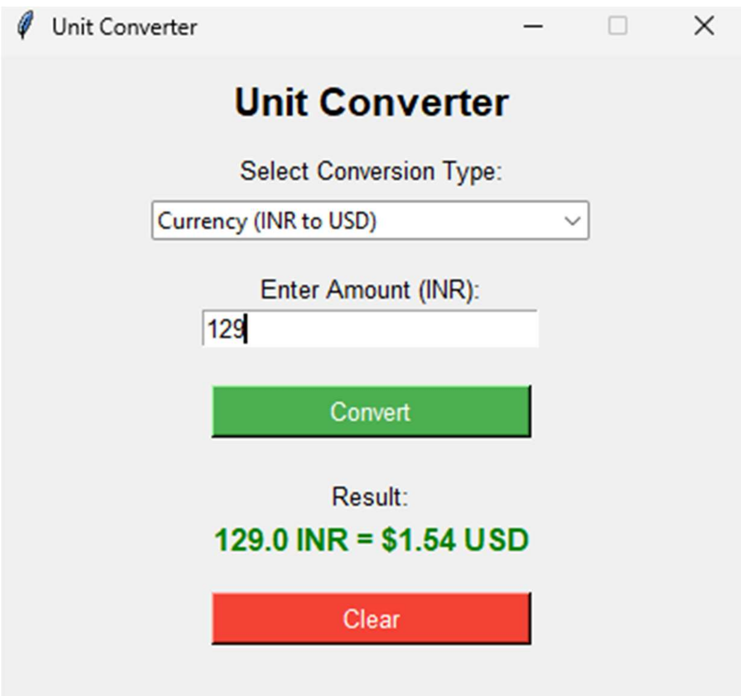
```
import tkinter as tk

from tkinter import ttk


class UnitConverterApp:

    def __init__(self, root):

        self.root = root

        self.root.title("Unit Converter")

        self.root.geometry("400x350")

        self.root.resizable(False, False)


        # Title

        title = tk.Label(root, text="Unit Converter", font=("Arial", 16, "bold"))

        title.pack(pady=10)


        # Conversion Type

        tk.Label(root, text="Select Conversion Type:", font=("Arial", 10)).pack()

        self.conversion_type = ttk.Combobox(root, values=["Currency (INR to USD)", "Temperature (°C to °F)", 
"Length (Inches to Feet)"], state="readonly", width=35)

        self.conversion_type.pack(pady=5)

        self.conversion_type.bind("<<ComboboxSelected>>", self.update_labels)


        # Input Frame
```

```
        input_frame = tk.Frame(root)
        input_frame.pack(pady=10)

        self.input_label = tk.Label(input_frame, text="Enter Value:", font=("Arial", 10))
        self.input_label.pack()

        self.input_field = tk.Entry(input_frame, width=25, font=("Arial", 10))
        self.input_field.pack()

        # Convert Button
        convert_btn = tk.Button(root, text="Convert", command=self.convert, bg="#4CAF50", fg="white", font=("Arial", 10), width=20)
        convert_btn.pack(pady=10)

        # Result Frame
        result_frame = tk.Frame(root)
        result_frame.pack(pady=10)

        self.result_label = tk.Label(result_frame, text="Result:", font=("Arial", 10))
        self.result_label.pack()

        self.result_field = tk.Label(result_frame, text="", font=("Arial", 12, "bold"), fg="green")
        self.result_field.pack()

        # Clear Button
        clear_btn = tk.Button(root, text="Clear", command=self.clear, bg="#f44336", fg="white", font=("Arial", 10), width=20)
        clear_btn.pack(pady=5)

    def update_labels(self, event=None):
        conversion = self.conversion_type.get()
        if conversion == "Currency (INR to USD)":
            self.input_label.config(text="Enter Amount (INR):")
```

```
        elif conversion == "Temperature (°C to °F)":
            self.input_label.config(text="Enter Temperature (°C):")
        elif conversion == "Length (Inches to Feet)":
            self.input_label.config(text="Enter Length (Inches):")

    def convert(self):
        try:
            value = float(self.input_field.get())
            conversion = self.conversion_type.get()

            if conversion == "Currency (INR to USD)":
                result = value / 83.5
                self.result_field.config(text=f"{value} INR = ${result:.2f} USD")
            elif conversion == "Temperature (°C to °F)":
                result = (value * 9/5) + 32
                self.result_field.config(text=f"{value}°C = {result:.2f}°F")
            elif conversion == "Length (Inches to Feet)":
                result = value / 12
                self.result_field.config(text=f"{value} Inches = {result:.2f} Feet")
            else:
                self.result_field.config(text="Please select a conversion type", fg="red")
        except ValueError:
            self.result_field.config(text="Invalid input! Enter a number", fg="red")

    def clear(self):
        self.input_field.delete(0, tk.END)
        self.result_field.config(text="", fg="green")
        self.conversion_type.set("")

if __name__ == "__main__":
    root = tk.Tk()
    app = UnitConverterApp(root)
    root.mainloop()
```







Conclusion

This program demonstrates how Tkinter can create simple GUI forms in Python, allowing user input, basic validation, and interactive feedback through message boxes in a user-friendly interface.